Fault-Tolerant Python:
Using tenacity, pybreaker, and asyncio for Cloud Resilience

PyCon JP 2026 · August 21–22, 2026 · International Conference Center Hiroshima

Shivam Chaurasia · Solution Architect 1 · EPAM Systems, India

---

1. Submission Metadata

Title	Fault-Tolerant Python: Using tenacity, pybreaker, and asyncio for Cloud Resilience
Speaker	Shivam Chaurasia, Solution Architect 1, EPAM Systems, India
Category	Application Development
Audience Level	Intermediate — Python async/await experience required. No prior resilience pattern knowledge assumed.
Duration	30 minutes (standard slot)
45-min option	No
Language	English (slides will include Japanese key terms)
Dependencies	tenacity 9.x · pybreaker 1.x · httpx · asyncio (stdlib)

---

2. Pretalx Form Fields

Abstract

Distributed systems fail. Networks partition, services time out, databases become temporarily unavailable. The question is not whether your Python service will encounter failures — it is whether it will handle them gracefully or cascade them to your users.

This talk gives you three production-grade tools: tenacity for smart retry with exponential backoff and jitter; pybreaker for circuit breaking that stops cascading failures before they snowball; and asyncio primitives for non-blocking, concurrent resilience patterns.

You will leave with working code for five real failure scenarios — transient HTTP errors, slow downstream services, connection pool exhaustion, partial degradation, and total dependency failure — and a decision framework for combining these tools correctly.

Your Experience With This Topic

As a Solution Architect at EPAM Systems, I design fault-tolerant Python services for enterprise cloud clients. I have debugged retry storms that brought down databases, cascading failures that turned a single outage into a full platform incident, and async deadlocks from misconfigured timeouts. Every pattern in this talk was shaped by those production incidents — not by reading library documentation in a quiet room.

What Discussions Can You Have With Attendees?

I want to hear: What failure has brought your Python service down at the worst moment? Does your team currently retry with sleep() and a for loop? Have you experienced a retry storm or a cascading failure in production? These war stories are what a conference room provides that no documentation can — and the patterns I show become far more memorable when mapped to real incidents from the audience.

---

3. Talk Outline (30 Minutes)

Time	Section	Content Note
0:00–3:00	Section 1 — The Cost of Silence	Opening incident story; failure taxonomy (transient / slow / total)
3:00–6:00	Section 2 — Why Naive Retry is Dangerous	Retry storm diagram; thundering herd; sleep() anti-pattern
6:00–12:00	Section 3 — tenacity: Smart Retry	wait_exponential + jitter; retry_if_exception_type; before_sleep hooks
12:00–18:00	Section 4 — pybreaker: Circuit Breaking	Closed/Open/Half-Open state machine; listener for metrics; fallback
18:00–23:00	Section 5 — asyncio: Non-Blocking Resilience	asyncio.wait_for() for timeouts; Semaphore as Bulkhead; async integration
23:00–27:00	Section 6 — The Resilience Stack	Composing all three: timeout → retry → circuit breaker → fallback
27:00–29:00	Section 7 — Decision Framework	Failure type → tool matrix; which to reach for first
29:00–30:00	Closing + Discussion Invite	Key message; QR code; hallway discussion prompt

---

4. Detailed Section Notes

Section 1 — The Cost of Silence [0:00–3:00]

- Opening story: a real-world incident where a single misconfigured retry multiplied load 50× and downed a downstream service.

- Failure taxonomy — three categories every cloud Python service encounters:
- **Transient:** Transient: network hiccup, DNS blip, momentary overload — safe to retry.
- **Slow:** Slow: dependency is responding but taking 10× normal time — needs timeout + circuit breaker.
- **Total:** Total: dependency is completely down — needs circuit breaker + fallback.
- Thesis: 'Your error handling is as important as your business logic.'

Section 2 — Why Naive Retry is Dangerous [3:00–6:00]

- Show the classic anti-pattern: for attempt in range(3): try: ... except: time.sleep(1)
- Retry storm: every client retries simultaneously on recovery — the thundering herd.
- Fixed sleep amplifies the problem; no jitter = all retries land at the same millisecond.
- No circuit breaker = retrying into an already-dead service, exhausting connection pools.
- Transition: 'Let's replace each of these problems with the right tool.'

Section 3 — tenacity: Smart Retry [6:00–12:00]

3a Exponential backoff with jitter [6:00–8:30]

- wait_exponential(multiplier=1, min=1, max=30) — doubles wait on each attempt up to a cap.
- wait_random_min_max() or wait_exponential_jitter() — spreads retries across time.
- Live code: @retry decorator wrapping an httpx.AsyncClient call.

3b Precise retry conditions [8:30–10:30]

- retry_if_exception_type(httpx.HTTPError) — only retry on network errors, not business errors.
- retry_if_result(lambda r: r.status_code >= 500) — retry on 5xx, not 4xx.
- Combining conditions: retry_if_exception_type(...) | retry_if_result(...)

3c Observability hooks [10:30–12:00]

- before_sleep=before_sleep_log(logger, logging.WARNING) — free structured logging per attempt.
- after=after_log(logger, logging.INFO) — logs final outcome.
- Statistics: fn.retry.statistics for last attempt count, delay, outcome.

Section 4 — pybreaker: Circuit Breaking [12:00–18:00]

4a The state machine [12:00–14:30]

- CLOSED: normal operation; failures counted. Opens when fail_max reached.
- OPEN: all calls immediately raise CircuitBreakerError — fast-fail, no waiting.
- HALF-OPEN: after reset_timeout, one probe call allowed; success → CLOSED, failure → OPEN.
- Live code: CircuitBreaker(fail_max=5, reset_timeout=30) wrapping a payment service call.

4b Listeners and metrics [14:30–16:30]

- CircuitBreakerListener — hook into state transitions for Prometheus / CloudWatch metrics.
- Live code: custom listener that increments a counter on state change.
- Show how to check breaker.current_state in a health endpoint.

4c Combining pybreaker with tenacity [16:30–18:00]

- Key rule: retry wraps circuit breaker — retrying a CLOSED breaker is fine; never retry an OPEN one.
- retry_if_not_exception_type(pybreaker.CircuitBreakerError) — stops tenacity retrying when circuit is open.

Section 5 — asyncio: Non-Blocking Resilience [18:00–23:00]

5a asyncio.wait_for() — timeouts [18:00–20:00]

- asyncio.wait_for(coro, timeout=2.0) — raises asyncio.TimeoutError after deadline.
- Wrapping at the right granularity: per-request timeout, not per-retry-attempt.
- Live code: async outer function combining wait_for + tenacity retry.

5b asyncio.Semaphore — Bulkhead [20:00–22:00]

- Bulkhead pattern: limit concurrent calls to a single dependency to prevent pool exhaustion.
- async with semaphore: limits inflight calls; callers wait, not fail.
- Live code: shared Semaphore(10) protecting all calls to a third-party API.

5c asyncio.gather with return_exceptions [22:00–23:00]

- Fan-out calls: asyncio.gather(*calls, return_exceptions=True).
- Handle partial failures without cancelling successful sibling tasks.

Section 6 — The Resilience Stack [23:00–27:00]

The correct composition order (outer → inner):

1. **1.** Timeout (asyncio.wait_for) — hard wall on total time.
2. **2.** Retry (tenacity) — retry transient errors with backoff.
3. **3.** Circuit Breaker (pybreaker) — fast-fail when dependency is unhealthy.
4. **4.** Bulkhead (asyncio.Semaphore) — limit concurrent calls.
5. **5.** Fallback — return cached/default value when all else fails.

Live code: full composition wrapping a real external API call — all five layers, ~40 lines.

- Anti-pattern: retrying after circuit is OPEN — show why retry must not catch CircuitBreakerError.
- Anti-pattern: per-attempt timeout vs per-call timeout — show the difference.

Section 7 — Decision Framework [27:00–29:00]

Three questions to ask for any dependency call:

6. Is this failure transient (< 1 s)? → tenacity with short exponential backoff.
7. Is this dependency slow or intermittently down? → pybreaker + reset_timeout.
8. Are we making many concurrent calls? → asyncio.Semaphore bulkhead.

Full matrix in Section 9.

Section 8 — Closing + Discussion Invite [29:00–30:00]

- Recap: three tools, five failure scenarios, one composable resilience stack.
- Key message: 'A well-configured tenacity + pybreaker + asyncio stack turns a 3 AM incident into a logged warning.'
- QR code → companion repository with all runnable examples.
- Discussion prompt: 'What is the one failure scenario in your system you have been afraid to test?'

---

5. Code Samples (Live Demos — Python 3.12+)

5.1 Naive retry — the anti-pattern (shown first, then replaced)

```
# ❌ ANTI-PATTERN — do not use
import time, httpx

def fetch_user_naive(user_id: int) -> dict:
    for attempt in range(3):
        try:
            return httpx.get(f"https://api.example.com/users/{user_id}").json()
        except httpx.HTTPError:
            time.sleep(1)           # fixed sleep — all retries land at same ms
    raise RuntimeError("Failed after 3 attempts")

# Problems:
#  1. No jitter  → thundering herd on recovery
#  2. Retries 4xx errors  → amplifies bad requests
#  3. No circuit breaker  → retries into a dead service until timeout
#  4. Blocks the thread  → starvation under asyncio
```

5.2 tenacity — exponential backoff, jitter, typed retry conditions

```
import httpx
import logging
from tenacity import (
    retry, stop_after_attempt,
    wait_exponential_jitter,
    retry_if_exception_type,
    before_sleep_log, after_log,
)

logger = logging.getLogger(__name__)

@retry(
    stop=stop_after_attempt(4),
    wait=wait_exponential_jitter(initial=1, max=30),   # doubles + random jitter
    retry=retry_if_exception_type(httpx.HTTPStatusError)
          | retry_if_exception_type(httpx.ConnectError),
    before_sleep=before_sleep_log(logger, logging.WARNING),
    after=after_log(logger, logging.INFO),
    reraise=True,
)
async def fetch_user(user_id: int) -> dict:
    async with httpx.AsyncClient(timeout=5.0) as client:
        response = await client.get(f"https://api.example.com/users/{user_id}")
        response.raise_for_status()   # raises HTTPStatusError on 4xx/5xx
        return response.json()

# Only 5xx / connection errors are retried — 4xx pass through immediately.
# Jitter ensures retries are staggered across clients.
```

5.3 pybreaker — circuit breaker with metrics listener

```
import pybreaker
import httpx
import logging
```

```
logger = logging.getLogger(__name__)


class LoggingListener(pybreaker.CircuitBreakerListener):
    """Hook into state transitions for structured logging / metrics."""
    def state_change(self, cb, old_state, new_state):
        logger.warning(
            "Circuit '%s' changed: %s → %s",
            cb.name, old_state.name, new_state.name,
        )
    def failure(self, cb, exc):
        logger.error("Circuit '%s' recorded failure: %s", cb.name, exc)


payment_breaker = pybreaker.CircuitBreaker(
    fail_max=5,             # open after 5 consecutive failures
    reset_timeout=30,       # probe again after 30 s
    name="payment-service",
    listeners=[LoggingListener()],
)

@payment_breaker
def call_payment(order_id: int) -> dict:
    response = httpx.post(
        "https://payments.internal/charge",
        json={"order_id": order_id},
        timeout=3.0,
    )
    response.raise_for_status()
    return response.json()

# When the circuit is OPEN, call_payment raises CircuitBreakerError immediately
# — no HTTP call is made, connection pool is preserved.
```

5.4 asyncio — timeout + Semaphore bulkhead

```
import asyncio
import httpx

# Bulkhead: max 10 concurrent calls to this dependency
_semaphore = asyncio.Semaphore(10)

async def fetch_product(product_id: int) -> dict:
    async with _semaphore:                    # wait here if 10 calls already inflight
        try:
            return await asyncio.wait_for(    # hard 3-second wall-clock timeout
                _do_fetch(product_id),
                timeout=3.0,
            )
        except asyncio.TimeoutError:
            raise TimeoutError(f"Product {product_id} fetch exceeded 3 s")

async def _do_fetch(product_id: int) -> dict:
    async with httpx.AsyncClient() as client:
        r = await client.get(f"https://catalog.internal/products/{product_id}")
        r.raise_for_status()
```

```
        return r.json()


# Fan-out: partial failure doesn't cancel successful calls
async def fetch_products(ids: list[int]) -> list[dict | Exception]:
    return await asyncio.gather(
        *[fetch_product(pid) for pid in ids],
        return_exceptions=True,              # exceptions returned, not raised
    )
```

5.5 The Resilience Stack — all five layers composed

```
import asyncio, httpx, pybreaker, logging
from tenacity import (
    retry, stop_after_attempt, wait_exponential_jitter,
    retry_if_exception_type, retry_if_not_exception_type,
)

logger = logging.getLogger(__name__)
_breaker   = pybreaker.CircuitBreaker(fail_max=5, reset_timeout=30, name="orders")
_semaphore = asyncio.Semaphore(20)

@retry(
    stop=stop_after_attempt(3),
    wait=wait_exponential_jitter(initial=0.5, max=10),
    # ✅ retry transient HTTP errors
    retry=retry_if_exception_type(httpx.HTTPStatusError),
    # ✅ but NOT when the circuit is open — fast-fail immediately
    retry=retry_if_not_exception_type(pybreaker.CircuitBreakerError),
)
async def get_order(order_id: int) -> dict:
    """Layer 1-4: timeout → retry → circuit breaker → bulkhead."""
    async with _semaphore:                               # 4. Bulkhead
        try:
            return await asyncio.wait_for(               # 1. Timeout
                _fetch_order(order_id), timeout=4.0
            )
        except asyncio.TimeoutError:
            raise httpx.ReadTimeout("order service timed out", request=None)

@_breaker                                                # 3. Circuit Breaker
async def _fetch_order(order_id: int) -> dict:
    async with httpx.AsyncClient() as client:
        r = await client.get(f"https://orders.internal/{order_id}")
        r.raise_for_status()                             # 2. surfaces for retry
        return r.json()

async def get_order_with_fallback(order_id: int) -> dict:
    """Layer 5: Fallback — return cached/default on total failure."""
    try:
        return await get_order(order_id)
    except (pybreaker.CircuitBreakerError, httpx.HTTPError, TimeoutError):
        logger.warning("Returning fallback for order %s", order_id)
        return {"order_id": order_id, "status": "unknown", "source": "fallback"}
```

5.6 Health endpoint exposing circuit breaker state

```
from fastapi import FastAPI
import pybreaker

app = FastAPI()

# Assume _breaker is imported from your resilience module
@app.get("/health")
async def health() -> dict:
    return {
        "status": "ok",
        "circuit_breakers": {
            "orders": _breaker.current_state,
            "payments": payment_breaker.current_state,
        },
    }
# Returns: {"status":"ok","circuit_breakers":{"orders":"closed","payments":"open"}}
# Kubernetes liveness probe can 503 when any critical breaker is "open".
```

---

6. Slide Structure (22 Slides)

#	Slide Title	Content Note
1	Title Slide	Title, speaker, EPAM Systems, PyCon JP 2026, QR code
2	The Incident	Opening story: retry storm that downed a DB at 3 AM
3	Failure Taxonomy	Visual: three lanes — Transient / Slow / Total
4	The Naive Retry	Code: for loop + time.sleep — 'How many of you have written this?'
5	The Thundering Herd	Diagram: all clients retrying simultaneously on recovery
6	Meet the Stack	Visual: tenacity + pybreaker + asyncio logos with their role
7	tenacity — Exponential Backoff	wait_exponential_jitter graph; code snippet
8	tenacity — Retry Conditions	retry_if_exception_type vs retry_if_result; 4xx pass-through
9	tenacity — Observability	before_sleep_log code; statistics dict output
10	Circuit Breaker — State Machine	Large diagram: Closed ↔ Open ↔ Half-Open with triggers
11	pybreaker — Code	CircuitBreaker() + listener code
12	pybreaker — Health Endpoint	FastAPI /health returning breaker state; Kubernetes probe
13	tenacity + pybreaker — Composition Rule	Key rule: retry must NOT catch CircuitBreakerError
14	asyncio.wait_for() — Timeout	Code: timeout=4.0 wrapping a coroutine
15	asyncio.Semaphore — Bulkhead	Diagram: 10 slots; code: async with semaphore
16	asyncio.gather — Partial Failure	return_exceptions=True; handling

		mixed results
17	The Resilience Stack — Visual	Layered diagram: timeout → retry → breaker → bulkhead → fallback
18	The Resilience Stack — Code	Full 40-line composition from Section 5.5
19	Fallback Strategies	Cache / default / degraded response; when to use each
20	Decision Framework	Failure type → tool matrix — large, photographable
21	Resources + Libraries	tenacity docs, pybreaker GitHub, asyncio docs; 3 books; QR code
22	Q&A + Discussion Prompt	'What failure are you afraid to test in production?'

7. Review Criteria Self-Assessment (PyCon JP 2026)

Criterion	How this proposal addresses it
Firsthand experience	Shivam has designed fault-tolerant Python services at EPAM Systems across multiple enterprise cloud engagements. The opening incident story and every anti-pattern highlighted are drawn from real production failures he has debugged or prevented.
Originality	Most retry/resilience talks cover one library. This talk shows how all three tools compose into a single stack with a clear layering rule — and demonstrates the non-obvious pitfall of retrying a CircuitBreakerError. The health endpoint integration with Kubernetes liveness probes is a practical production detail rarely covered.
Clarity	Structured around failure types (transient / slow / total) that map directly onto the tools. Each demo is the minimal code that proves the point. The decision matrix fits on one slide attendees can photograph and apply immediately.
Interactivity	Opens by asking 'Who has written for attempt in range(3): time.sleep(1)?' — virtually every Python developer has. Closes with 'What failure are you afraid to test?' — a question that surfaces incidents teams haven't discussed openly.
Relevance	Cloud-native Python services are the dominant Python deployment pattern in 2026. Every attendee building with FastAPI, Django, or async workers faces these failure modes. The three libraries covered are all pure Python, pip-installable, and production-proven.

8. Resilience Tools Overview

Tool / Pattern	Failure Type Addressed	Key API / Concept
tenacity	Transient failures (retry)	wait_exponential_jitter,

		retry_if_exception_type, before_sleep_log
pybreaker	Unhealthy dependencies (breaker)	CircuitBreaker(fail_max, reset_timeout), listeners, current_state
asyncio.wait_for	Slow dependencies (timeout)	Hard deadline per call; raises asyncio.TimeoutError
asyncio.Semaphore	Concurrent overload (bulkhead)	Limit inflight calls; async with semaphore context manager
asyncio.gather	Fan-out partial failures	return_exceptions=True — mixed success/failure results
Fallback function	Total dependency failure	Cached value / degraded response / default; last line of defence

---

9. Fault-Tolerance Decision Framework (Final Slide)

For every external dependency call, ask these three questions in order:

Question	Tool	Recommended Configuration
Is the error transient (network blip, momentary overload)?	tenacity	wait_exponential_jitter + retry_if_exception_type; 3-4 attempts max
Is the dependency slow or intermittently failing?	pybreaker	fail_max=5, reset_timeout=30; fast-fail when OPEN
Am I making many concurrent calls?	asyncio.Semaphore	Bulkhead with 10-20 slots per dependency
Can I tolerate a hard time boundary?	asyncio.wait_for	Per-request timeout; wrap the full retry+breaker stack
What should I return when everything fails?	Fallback	Cached response, degraded result, or explicit error — never silence
Combining all of the above?	Resilience Stack	Timeout → Retry → Circuit Breaker → Bulkhead → Fallback (outer → inner)

Critical composition rule:

- ✅ retry_if_not_exception_type(CircuitBreakerError) — tenacity must never retry an open circuit.
- ✅ Never catch and swallow CircuitBreakerError silently — always provide a fallback or re-raise.
- ✅ Per-attempt timeout (inside retry) is different from per-call timeout (outside retry) — use both.

---

10. Anti-Patterns to Avoid

Anti-Pattern	Why It Hurts and How to Fix It
Fixed sleep() retry	All clients retry at the same moment. Replace with wait_exponential_jitter().
Retrying 4xx errors	400 Bad Request will never succeed. Use retry_if_exception_type to whitelist retryable errors.
Retrying a OPEN circuit	Retrying CircuitBreakerError defeats the purpose. Use retry_if_not_exception_type(CircuitBreakerError).

Timeout only on the entire retry stack	A 3-attempt retry with a 10 s total timeout can give each attempt 10 s. Set per-attempt timeout too.
No fallback on CircuitBreakerError	Fast-failing is not enough — the caller still needs a response. Always pair a breaker with a fallback.
Unbounded Semaphore concurrency	Semaphore(1000) with 1000 tasks provides no bulkhead. Size it to the dependency's max capacity.
Circuit breaker with no metrics	A silent circuit breaker is invisible. Wire a listener to your logging/metrics pipeline from day one.

11. References & Further Reading

tenacity documentation	tenacity.readthedocs.io
pybreaker GitHub	github.com/danielfm/pybreaker
Python asyncio documentation	docs.python.org/3/library/asyncio.html
Release It! (2nd Ed.)	Michael T. Nygard — Pragmatic Programmers 2018
Building Microservices (2nd Ed.)	Sam Newman — O'Reilly 2021
Designing Data-Intensive Applications	Martin Kleppmann — O'Reilly 2017
httpx documentation	www.python-httpx.org
Cloud Design Patterns	Microsoft Azure Architecture Center (free online)